

Missing Semester 2026

课程概览 + Shell 入门

练习题解答

运行环境: macOS + zsh (bash 兼容)

2026 年 8 月 31 日

摘要

本文档为 Missing Semester 2026 第一讲「课程概览 + Shell 入门」的课后练习题完整解答。所有命令均在终端中实际运行验证，包含命令、输出及原理解释。

目录

1 基础命令与重定向

题 1：确认 Shell 环境

要求：本课程要求使用类 Unix Shell (Bash 或 ZSH)。运行 `echo $SHELL` 确认。

解答：

```
echo $SHELL
```

实际输出：

```
/bin/zsh
```

说明：当前默认 Shell 是 zsh, 属于类 Unix Shell, 符合课程要求。bash 也可用 (版本 3.2.57)。Windows 用户需使用 WSL 或 Linux 虚拟机, 不能用 `cmd.exe` 或 PowerShell。

题 2：ls -l 的输出格式

要求：ls -l 的 -l 选项作用是什么？运行 `ls -l /`, 每一行最前面的 10 个字符分别代表什么？

解答：

```
ls -l /
```

实际输出 (节选)：

```
total 10
drwxrwxr-x 42 root  admin  1344 Aug 30 13:55 Applications
drwxr-xr-x@ 39 root  wheel  1248 Apr  6 16:10 bin
lrwxr-xr-x@  1 root  wheel    11 Apr  6 16:10 etc -> private/etc
```

-l 选项作用：以长格式 (long format) 列出文件, 显示文件类型、权限、硬链接数、所有者、所属组、大小、修改时间和文件名。

前 10 个字符的含义：

位置	含义	示例
第 1 位	文件类型	d= 目录, l= 符号链接, -= 普通文件
第 2-4 位	所有者 (user) 权限	rw- = 读、写、执行
第 5-7 位	所属组 (group) 权限	r-x = 读、执行 (无写权限)
第 8-10 位	其他用户 (others) 权限	r-x = 读、执行

例如 `drwxr-xr-x` 表示：这是一个目录, 所有者可读写执行, 组用户可读执行, 其他用户可读执行。

注：第 11 位可能是 `@` (macOS 扩展属性) 或 `+` (ACL), 不属于权限位。

题 3: glob 模式匹配

要求：什么是 glob? 测试 `ls *.txt`、`ls file?.txt`、`ls {a,b,c}.txt` 等模式。

glob 是什么：glob 是 Shell 提供的文件名通配符模式匹配机制。Shell 在执行命令前，会先将包含通配符的参数展开为匹配的文件名列表，再传给程序。

模式	含义
*	匹配任意长度的任意字符（包括空）
?	匹配恰好一个任意字符
[abc]	匹配 a、b、c 中任意一个字符
[a-z]	匹配 a 到 z 范围内任意一个字符
{a,b,c}	大括号扩展，生成 a、b、c 多个字符串

实际测试：

```
mkdir glob_test && cd glob_test
touch file1.txt file2.txt file3.txt a.txt b.txt c.txt abc.txt ab.txt

ls *.txt          # 匹配所有 .txt 文件
ls file?.txt      # 匹配 file + 任意一个字符 + .txt
ls {a,b,c}.txt    # 大括号扩展为 a.txt b.txt c.txt
ls [ab]*.txt      # 匹配以 a 或 b 开头的 .txt 文件
```

输出：

```
=== ls *.txt ===
a.txt  ab.txt  abc.txt  b.txt  c.txt  file1.txt  file2.txt  file3.txt

=== ls file?.txt ===
file1.txt  file2.txt  file3.txt

=== ls {a,b,c}.txt ===
a.txt  b.txt  c.txt

=== ls [ab]*.txt ===
a.txt  ab.txt  abc.txt  b.txt
```

题 4: 三种引号的区别

要求：' 单引号'、" 双引号" 和 `$'ANSI 引号'` 有什么区别? 写一条命令，输出同时包含字面量 `$`、`!` 和换行符的字符串。

实际测试：

```
# 单引号：完全字面量
echo 'It costs $100! Hello\nWorld'
```

引号类型	变量展开	转义序列	历史扩展	适用场景
'...' 单引号	否	否	否	纯字面量字符串
"..." 双引号	是	否 (除少数)	可能触发	需要嵌入变量
\$'...' ANSI-C	否	是	否	需要转义序列的字面量

```
# 输出: It costs $100! Hello\nWorld

# 双引号: 展开变量, \n 不解释
price=100
echo "It costs $price! Hello\nWorld"
# 输出: It costs $price! Hello\nWorld

# ANSI-C 引号: 解释 \n, $ 是字面量
echo '$It costs $100! Hello\nWorld'
# 输出:
# It costs $100! Hello
# World
```

题目要求的命令 (同时包含字面量 \$、! 和换行符):

```
echo '$价格是 $100!\n这是第二行'
```

输出:

```
价格是 $100!
这是第二行
```

也可以用 `printf '价格是 $100!\n 这是第二行\n'`, `printf` 默认就解释转义序列。

题 5: 标准流与重定向

要求: Shell 有三条标准流: `stdin(0)`、`stdout(1)`、`stderr(2)`。运行 `ls /nonexistent /tmp`, 把 `stdout` 和 `stderr` 分别重定向到两个文件。如何把两者都重定向到同一个文件?

三条标准流:

- `stdin (0)`: 标准输入, 默认来自键盘
- `stdout (1)`: 标准输出, 默认打印到终端
- `stderr (2)`: 标准错误, 默认也打印到终端 (独立于 `stdout`)

分别重定向到两个文件:

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
```

- `> stdout.txt` 等价于 `1> stdout.txt`, 把标准输出写入 `stdout.txt`

- `2> stderr.txt` 把标准错误写入 `stderr.txt`

实际输出:

```
stdout.txt: 包含 /tmp 目录下的文件列表
stderr.txt: 包含 "ls: /nonexistent: No such file or directory"
```

两者重定向到同一个文件 (两种方法):

方法 1 (POSIX 标准, 推荐):

```
ls /nonexistent /tmp > both.txt 2>&1
```

- `> both.txt` 先把 `stdout` 指向 `both.txt`
- `2>1` 把 `stderr` 重定向到当前 `stdout` 指向的地方 (即 `both.txt`)
- 顺序很重要: `2>1` 必须在 `>` 之后

方法 2 (bash/zsh 简写):

```
ls /nonexistent /tmp &> both.txt
```

题 6: 退出状态与 `&&` / `||`

要求: `$?` 保存上一条命令的退出状态 (0= 成功)。&& 仅在前一条成功时执行后一条; `||` 仅在前一条失败时执行后一条。写一行命令: 仅当 `/tmp/mydir` 不存在时才创建它。

退出状态: 每个命令执行后返回一个 0-255 的整数, 0 表示成功, 非 0 表示失败。可用 `echo $?` 查看。

逻辑运算符:

- `cmd1 && cmd2`: `cmd1` 成功 (退出码 0) 才执行 `cmd2`
- `cmd1 || cmd2`: `cmd1` 失败 (退出码非 0) 才执行 `cmd2`
- 两者可组合: `cmd1 && cmd2 || cmd3` (类似三元运算符)

题目要求的命令:

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

解释:

- `[ -d /tmp/mydir ]` 测试目录是否存在 (`test -d` 的简写)
- 如果存在, `[` 命令返回 0 (成功), `||` 后面的 `mkdir` 不执行
- 如果不存在, `[` 返回非 0 (失败), `||` 触发 `mkdir /tmp/mydir`

实际验证:

```
# 第一次运行：目录不存在，创建成功
[ -d /tmp/mydir ] || mkdir /tmp/mydir
ls -ld /tmp/mydir
# 输出：drwxr-xr-x  2 liuchenxi  wheel  64 Aug 30 16:51 /tmp/mydir

# 第二次运行：目录已存在，不重复创建
[ -d /tmp/mydir ] || mkdir /tmp/mydir
# 无输出，无报错
```

更简洁的等价写法：`mkdir -p /tmp/mydir`（`-p` 表示目录已存在时不报错），但题目要求用 `&&` 逻辑。

2 脚本编写与权限

题 7：为什么 `cd` 必须是 Shell 内建命令？

要求：为什么 `cd` 必须是 Shell 内建命令，而不能是独立程序？（提示：想想子进程能影响和不能影响父进程的哪些状态。）

核心原因：子进程无法修改父进程的工作目录。

详细解释：

1. 进程独立性：在 Unix 系统中，每个进程都有自己独立的「当前工作目录」（`cwd`）。当 Shell 执行一个外部程序时，会通过 `fork()` 创建子进程，再用 `exec()` 加载程序。子进程继承父进程的 `cwd`，但这只是一个副本。
2. 如果 `cd` 是独立程序：
 - Shell `fork` 出子进程
 - 子进程执行 `cd /some/path`，修改的是子进程自己的 `cwd`
 - 子进程结束后，Shell（父进程）的 `cwd` 完全没有改变
 - 结果就是：你执行了 `cd /some/path`，但提示符显示的当前目录没变
3. 内建命令的优势：内建命令由 Shell 自身执行，不创建子进程，可以直接修改 Shell 进程自己的 `cwd`，从而真正改变后续命令的工作目录。

验证：

```
which cd
# 输出：cd: shell built-in command (或 no cd in $PATH)
# 说明 cd 不是独立程序，而是 Shell 内建命令
```

同类内建命令：`cd`、`export`、`alias`、`exit`、`source/. 等`——凡是需要修改 Shell 自身状态的命令，都必须是内建命令。

题 8：写脚本检查文件是否存在

要求：写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在，并根据结果输出不同提示。

脚本 `check.sh`：

```
#!/bin/bash
# 检查文件是否存在的脚本

if [ -f "$1" ]; then
    echo "文件 '$1' 存在。"
else
    echo "文件 '$1' 不存在。"
fi
```

关键点：

- \$1 是第一个命令行参数
- `[ -f "$1" ]` 测试 \$1 是否为存在的普通文件 (`-f = file`)
- "\$1" 加双引号防止文件名含空格时出错
- `if ...; then ... else ... fi` 是 bash 条件语句结构

其他常用测试条件：

条件	含义
<code>-f file</code>	存在且是普通文件
<code>-d file</code>	存在且是目录
<code>-e file</code>	存在（任何类型）
<code>-r file</code>	存在且可读
<code>-w file</code>	存在且可写
<code>-x file</code>	存在且可执行
<code>-s file</code>	存在且大小大于 0

题 9：`chmod +x` 的作用

要求：把上一题的脚本保存为 `check.sh`。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？

`chmod` 前运行：

```
./check.sh somefile
# 输出: bash: ./check.sh: Permission denied
```

`chmod` 前的权限：

```
-rw----- 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

rw- = 所有者可读、可写，但不可执行

执行 `chmod +x check.sh` 后：

```
-rwx--x--x 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

rwx = 所有者可读、可写、可执行

`chmod` 后运行：

```
./check.sh somefile
# 输出：文件 'somefile' 不存在。

./check.sh check.sh
# 输出：文件 'check.sh' 存在。
```

为什么必须 `chmod +x`：

- Unix 系统中，文件能否被直接执行取决于可执行权限位（x 位），而不是文件扩展名
- 新建文件默认没有执行权限（安全考虑）
- `chmod +x` 给所有用户添加执行权限，Shell 才能通过 `./check.sh` 直接运行它
- 没有执行权限时，仍可通过解释器显式运行：`bash check.sh somefile`

shebang 行（`#!/bin/bash`）只在文件有执行权限且被直接运行时才生效。

题 10: `set -x` 的作用

要求：在脚本的 `set` 选项里加入 `-x` 会发生什么？写个简单脚本试试。

`set -x` 的作用：开启调试模式（xtrace）。Shell 在执行每条命令前，会先把该命令及其展开后的参数打印到 `stderr`（通常以 `+` 开头），方便追踪脚本执行流程。

演示脚本 `debug_demo.sh`：

```
#!/bin/bash
set -x

echo "开始执行脚本"
name="Shell"
echo "你好, $name"
for i in 1 2 3; do
    echo "第 $i 次循环"
done
echo "脚本结束"
```

运行输出（`+` 开头的行是 `set -x` 打印的追踪信息）：


```

+ echo '开始执行脚本'
开始执行脚本
+ name=Shell
+ echo '你好, Shell'
你好, Shell
+ i=1
+ echo '第 1 次循环'
第 1 次循环
+ i=2
+ echo '第 2 次循环'
第 2 次循环
+ i=3
+ echo '第 3 次循环'
第 3 次循环
+ echo '脚本结束'
脚本结束

```

常用 set 选项总结：

选项	作用
set -e	任何命令失败时立即退出脚本
set -u	使用未定义变量时报错（而非当作空字符串）
set -o pipefail	管道中任一命令失败，整个管道视为失败
set -x	打印每条执行的命令（调试用）

推荐组合：set -euo pipefail（脚本开头的「严格模式」），调试时再加 -x。

题 11：带日期的备份文件

要求：写一条命令，把文件复制为带当天日期的备份文件名（例如 notes.txt → notes_2026-01-12.txt）。
提示：\$(date +%Y-%m-%d)

命令：

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

原理：

- \$(date +%Y-%m-%d) 是命令替换：执行 date 命令，用其输出替换整个 \$()
- +%Y-%m-%d 是 date 的格式参数：%Y= 四位年，%m= 两位月，%d= 两位日
- 双引号确保日期展开后作为一个整体参数

实际运行：

```
echo "这是笔记内容" > notes.txt
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ls -l notes*.txt
```

输出:

```
-rw-r--r--  1 staff  19 Aug 30 16:52 notes.txt
-rw-r--r--  1 staff  19 Aug 30 16:52 notes_2026-08-30.txt
```

通用公式:

```
cp "文件名" "文件名_$(date +%Y-%m-%d).扩展名"
```

其他常用日期格式:

```
date +%Y%m%d           # 20260830
date +%Y-%m-%d_%H%M%S  # 2026-08-30_165200 (带时间)
date -v-1d +%Y-%m-%d   # 昨天的日期 (macOS)
```

题 12: 修改 flaky test 脚本接收命令行参数

要求: 修改讲义中的「复现偶尔才会失败的测试」脚本, 使它能够从命令行参数接收测试命令, 而不是写死 `cargo test my_test`。提示: `$1` 或 `$@`

修改后的脚本 `flaky_test.sh`:

```
#!/bin/bash
set -euo pipefail

# 从命令行参数接收测试命令, 而不是写死
TEST_CMD="$@"

if [ -z "$TEST_CMD" ]; then
    echo "用法: $0 <测试命令>"
    echo "示例: $0 cargo test my_test"
    exit 1
fi

# Start CPU stress in background
stress --cpu 8 &
STRESS_PID=$!

# Setup log file
LOGFILE="test_runs_$(date +%s).log"
echo "Logging to $LOGFILE"
echo "运行测试命令: $TEST_CMD"
```

```
# Run tests until one fails
RUN=1
while $TEST_CMD > "$LOGFILE" 2>&1; do
    echo "Run $RUN passed"
    ((RUN++))
done

# Cleanup and report
kill $STRESS_PID
echo "Test failed on run $RUN"
echo "Last 20 lines of output:"
tail -n 20 "$LOGFILE"
echo "Full log: $LOGFILE"
```

关键改动：

- 原脚本写死：while cargo test my_test > "\$LOGFILE" 2>&1; do
- 改为：TEST_CMD="\$@" 接收所有命令行参数，然后 while \$TEST_CMD > ...

特殊参数说明：

参数	含义
\$1	第一个参数
\$@	所有参数，每个参数独立（加引号后"\$@" 保留空格）
\$*	所有参数，用 IFS 连接成一个字符串
\$	参数个数
\$0	脚本自身名称
\$!	最近一个后台进程的 PID

使用方式：

```
./flaky_test.sh cargo test my_test    # 传 cargo test 命令
./flaky_test.sh pytest tests/         # 传 pytest 命令
./flaky_test.sh false                 # 用 false 测试（第一次就失败）
```

注意：\$@ 比 \$1 更适合接收完整命令，因为命令可能包含多个参数（如 cargo test my_test 是三个参数）。

3 管道、工具与进阶

题 13：找出 home 目录最常见的 5 种文件扩展名

要求：使用管道找出 home 目录中最常见的 5 种文件扩展名。提示：组合 find、grep/sed/awk、sort、uniq -c、head

命令：

```
find ~ -type f 2>/dev/null \
| grep -oE '\.[^./]+$' \
| sort \
| uniq -c \
| sort -rn \
| head -n 5
```

逐步拆解:

1. `find -type f`: 递归列出 home 目录下所有普通文件
2. `2>/dev/null`: 把权限错误等 `stderr` 丢弃
3. `grep -oE '\.[^./]+$'`: 提取每行末尾的扩展名
 - `-o`: 只输出匹配部分
 - `-E`: 使用扩展正则
 - `\.[^./]+$`: 匹配 `.` + 一个或多个非 `.` 非 `/` 字符 + 行尾
4. `sort`: 排序 (相同扩展名排在一起)
5. `uniq -c`: 去重并统计每行出现次数
6. `sort -rn`: 按数字逆序排序 (出现次数多的在前)
7. `head -n 5`: 取前 5 个

实际输出:

```
11425 .py
9438 .js
6911 .pyc
5855 .md
3948 .json
```

等价的 `awk` 版本:

```
find ~ -type f 2>/dev/null | awk -F. '{print ".$NF"}' | sort | uniq -c | sort -rn |
head -5
```

题 14: `xargs` 结合 `find` 统计 `.sh` 文件行数

要求: `xargs` 把 `stdin` 的每一行转换为命令参数。结合 `find` 和 `xargs` (不要用 `find -exec`), 找出目录中所有 `.sh` 文件, 并用 `wc -l` 统计每个文件行数。加分项: 正确处理文件名中的空格。
提示: `-print0` 和 `-0`

基础版本:

```
find . -name "*.sh" | xargs wc -l
```

问题:如果文件名包含空格(如 my script.sh),xargs 会把它拆成两个参数 my 和 script.sh,导致错误。

正确处理空格的版本 (加分项):

```
find . -name "*.sh" -print0 | xargs -0 wc -l
```

原理:

- find -print0: 用 null 字符 (\0) 分隔文件名, 而不是换行符
- xargs -0: 以 null 字符作为分隔符读取输入
- 因为文件名中不可能包含 null 字符, 所以这种方式 100% 安全处理任何文件名 (含空格、换行、特殊字符)

实际输出:

```
1 ./dir with spaces/my script.sh
34 ./flaky_test.sh
8 ./check.sh
10 ./debug_demo.sh
53 total
```

xargs 常用参数:

参数	含义
-0	以 null 字符分隔输入 (配合 find -print0)
-n N	每次传递 N 个参数给命令
-I {}	用 {} 作为参数占位符
-P N	并行运行 N 个进程

题 15: curl 获取课程网站 HTML, 统计列出了多少讲

要求: 使用 curl 获取课程网站的 HTML, 通过 grep 统计列出了多少讲。提示: 找出每讲课程名称在 HTML 中的共性; 用 curl -s 关闭进度输出。

命令:

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| wc -l
```

逐步拆解:

1. `curl -s URL`: 静默模式获取网页 HTML (不显示进度条)
2. `grep -oE 'href="/2026/[a-z-]+/\"'`: 提取所有课程链接
 - 课程链接的共性: `href="/2026/课程名/"`
 - 课程名由小写字母和连字符组成
3. `sort -u`: 排序并去重 (`-u = unique`)
4. `wc -l`: 统计行数, 即课程讲数

实际输出:

```
9
```

列出各讲名称:

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/\"' \
| sort -u \
| sed 's|href="/2026/||;s|/||'
```

输出:

```
agentic-coding
beyond-code
code-quality
command-line-environment
course-shell
debugging-profiling
development-environment
shipping-code
version-control
```

共 9 讲, 与课程介绍中「九场 1 小时的讲座」一致。

题 16: jq 处理 JSON

要求: 用 `curl` 获取示例数据, 再用 `jq` 提取 `version` 大于 6 的人员姓名。提示: 先 `jq .` 看结构; 再试 `jq '.[0] | select(...) | .name'`

获取数据并查看结构:

```
curl -s https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json > data.json
jq '[0]' data.json
```

JSON 结构:

```
{
  "name": "Adeel Solangi",
```

```

    "language": "Sindhi",
    "id": "V590F92YF627HFY0",
    "bio": "Donec lobortis eleifend...",
    "version": 6.1
  }

```

顶层是一个数组，每个元素包含 name、language、id、bio、version 字段。

提取 `version > 6` 的人员姓名：

```
jq '[] | select(.version > 6) | .name' data.json
```

jq 语法拆解：

- `[]`：遍历数组中的每个元素
- `select(.version > 6)`：筛选 version 字段大于 6 的元素
- `.name`：提取 name 字段

统计数量：

```
jq '[] | select(.version > 6)] | length' data.json
# 输出: 89
```

部分输出：

```

"Adeel Solangi"
"Aamir Solangi"
"Adil Eli"
"George Mifsud"
"John Falzon"
...

```

jq 常用操作速查：

表达式	含义
<code>.</code>	整个输入
<code>.field</code>	取字段
<code>[]</code>	遍历数组
<code>select(cond)</code>	筛选满足条件的元素
<code>length</code>	数组长度或字符串长度
<code>keys</code>	对象的所有键
<code>map(expr)</code>	对数组每个元素应用 expr

题 17: awk 按列过滤并交换列

要求: 写一个 awk 命令: 只输出第二列大于 100 的行, 并交换第一列和第三列。测试数据:

```
printf 'a 50 x\nb 150 y\nc 200 z\n'
```

命令:

```
printf 'a 50 x\nb 150 y\nc 200 z\n' \
| awk '$2 > 100 { temp=$1; $1=$3; $3=temp; print }'
```

awk 语法拆解:

- `$2 > 100`: 模式 (条件), 只处理第二列大于 100 的行
- `{ temp=$1; $1=$3; $3=temp; print }`: 动作
 - `temp=$1`: 把第一列存到临时变量
 - `$1=$3`: 第一列替换为第三列的值
 - `$3=temp`: 第三列替换为原来的第一列
 - `print`: 打印修改后的整行

实际输出:

```
y 150 b
z 200 c
```

验证:

- 原始行 `a 50 x`: 第二列 50, 不大于 100, 被过滤
- 原始行 `b 150 y`: 第二列 `150 > 100`, 交换第一列 (b) 和第三列 (y) → `y 150 b`
- 原始行 `c 200 z`: 第二列 `200 > 100`, 交换第一列 (c) 和第三列 (z) → `z 200 c`

awk 核心概念:

- awk 按行处理, 每行自动按空白字符分成列
- `$1`, `$2`, `$3...` 引用各列, `$0` 表示整行
- `NR` = 当前行号, `NF` = 当前行列数
- `FS` = 输入分隔符 (默认空白), `-F`, 可设为逗号
- `BEGIN { }` 在处理前执行, `END { }` 在处理后执行

题 18: 拆解 SSH 日志管道 + 从历史记录找出最常用命令

要求: 拆解讲义中的 SSH 日志处理管道: 每一步分别做了什么? 然后仿照它构建一个管道, 从 `~/.bash_history` (或 `~/.zsh_history`) 中找出最常使用的 Shell 命令。

第一部分：拆解 SSH 日志管道

原始管道：

```
ssh myserver 'journalctl -u sshd -b-1 | grep "Disconnected from" \
| sed -E 's/.*Disconnected from .* user (.*?) [^ ]+ port.*/\1/' \
| sort | uniq -c \
| sort -nk1,1 | tail -n10 \
| awk '{print $2}' | paste -sd,
```

逐步拆解：

步骤	命令	作用
1	ssh myserver '...'	登录远程服务器，在远程执行引号内的命令
2	journalctl -u sshd -b-1	查看 sshd 服务在上一次启动时的系统日志
3	grep "Disconnected from"	筛选包含「断开连接」的日志行
4	sed -E 's/...user (.*?) ...//'	用正则提取用户名，(.*?) 捕获，\1 输出
5	sort	对用户名排序（相同名字排在一起）
6	uniq -c	去重并统计每个用户名出现次数
7	sort -nk1,1	按第一列（次数）数值排序（升序）
8	tail -n10	取最后 10 个，即出现次数最多的 10 个
9	awk '{print \$2}'	只输出第二列（用户名），去掉次数
10	paste -sd,	把多行合并为一行，用逗号分隔

最终输出示例：

```
postgres,mysql,oracle,dell,ubuntu,inspur,test,admin,user,root
```

第二部分：从历史记录找出最常用命令

命令 (zsh 用户)：

```
LC_ALL=C awk '{print $1}' ~/.zsh_history | sort | uniq -c | sort -rn | head -n 10
```

命令 (bash 用户)：

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -rn | head -n 10
```

逐步拆解：

1. awk '{print \$1}': 提取每行的第一个词（即命令名，去掉参数）
2. sort: 排序
3. uniq -c: 统计每个命令出现次数
4. sort -rn: 按次数逆序排序

5. `head -n 10`: 取前 10 个

实际输出 (`zsh_history`):

```
123 brew
25 cd
22 git
20 sudo
14 ls
9 echo
8 fc-match
7 which
7 find
6 mkdir
```

说明:

- `zsh` 的历史文件默认格式可能包含时间戳前缀 (例如: `1234567890:0;command`), 如果有, 需要先用 `sed 's/^:[0-9]*:[0-9]*;/'` 去掉前缀
- `LC_ALL=C` 防止遇到非 UTF-8 字节时 `awk/sed` 报错
- 最常用的命令是 `brew` (macOS 包管理器), 其次是 `cd`、`git`、`sudo`、`ls`

附录: 本练习涉及的脚本文件

文件	说明
<code>check.sh</code>	题 8/9: 检查文件是否存在的脚本
<code>debug_demo.sh</code>	题 10: 演示 <code>set -x</code> 调试模式
<code>flaky_test.sh</code>	题 12: 接收命令行参数的 <code>flaky test</code> 脚本

所有脚本均已添加可执行权限, 可直接运行。

— 全文完 —